LECTURE SERIES · DATABASE SYSTEMS

Concurrency, Deadlock & Recovery

Keeping Transactions Correct When They Run Together

Md. Rakib Trofder

Lecturer, Bangladesh Army University of Science & Technology

Serializability · 2PL · Lock-Based · Graph-Based · Deadlock · Deadlock Handling

DEADLOCK BASICS

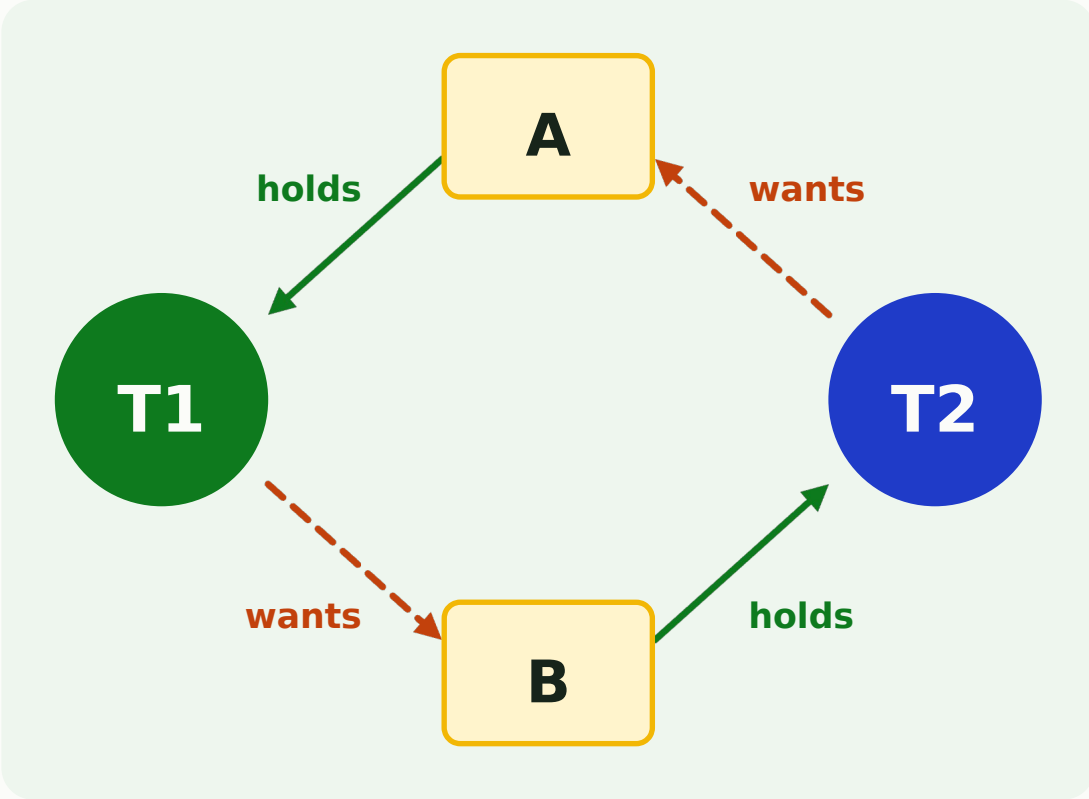
What Is a Deadlock?

A set of transactions where each one waits for another in the set — so none of them can ever continue.

REAL-LIFE PICTURE

Two cars meet on a one-lane bridge. Each waits for the other to reverse. Nobody moves — forever.

Solid arrow = holds the lock · **Dashed arrow** = waits for the lock



DEADLOCK BASICS

Three Ways to Handle Deadlock

Prevention

Design the rules so a deadlock can **never form**.

- Lock all items up front
- Lock items in a fixed order
- Wait-die / wound-wait

Detection & Recovery

Let it happen, **find it**, then fix it.

- Build a wait-for graph
- Look for a cycle
- Roll back a victim

Timeout

Waited **too long**? Roll it back.

- Very simple to build
- Hard to pick the limit
- May kill innocent waits

DEADLOCK DETECTION

How to Draw a Wait-For Graph

1. One **node** for every active transaction.
2. Draw **Ti → Tj** when Ti asks for a lock that Tj holds in a **conflicting** mode.
3. Erase the edge when Tj releases the lock and Ti gets it.
4. **Cycle in the graph ⇔ deadlock.** The DBMS checks for cycles from time to time.

WHEN DOES A REQUEST CONFLICT?

Held ↓ / Asked →	S (shared)	X (exclusive)
S held	OK — grant	Conflict — wait
X held	Conflict — wait	Conflict — wait

Only **S + S** can share an item. Every other pair makes the requester wait — and that wait is an edge.

EXAM Q7 (A) · STEP 1

Trace Every Lock Request

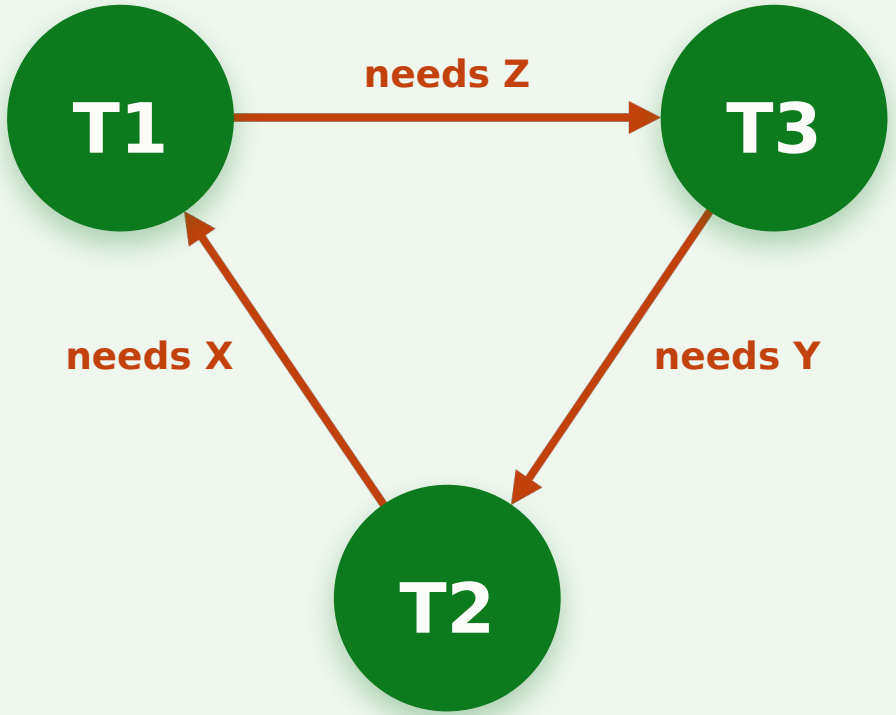
Read the schedule top to bottom. Only lock requests matter — Read steps just use a lock already held.

Step	Txn	Request	What happens
1	T1	Lock-S(X)	Granted — X is free
3	T3	Lock-X(Z)	Granted — Z is free
5	T1	Lock-X(Z)	Waits — T3 holds X(Z) → edge T1 → T3
6	T2	Lock-S(Y)	Granted — Y is free
8	T2	Lock-X(X)	Waits — T1 holds S(X) → edge T2 → T1
9	T3	Lock-X(Y)	Waits — T2 holds S(Y) → edge T3 → T2

LOCKS HELD AFTER STEP 9 `X: S by T1 · Y: S by T2 · Z: X by T3`

EXAM Q7 (A) · STEP 2

The Wait-For Graph — Is There a Cycle?



CYCLE FOUND

T1 → T3 → T2 → T1

YES — Deadlock!

- T1 waits for T3 (wants Z)
- T3 waits for T2 (wants Y)
- T2 waits for T1 (wants X)

Each one holds what the next one needs. **Nobody can ever move.**

EXAM Q7 (A) · STEP 3

Deadlock Recovery Procedure

STEP 1

Select a Victim

Pick the transaction that is cheapest to roll back: least work done, fewest locks, most work left, fewest others affected.

STEP 2

Roll It Back

Total: abort and restart it. **Partial:** undo only as far as needed to free the lock that breaks the cycle.

STEP 3

Avoid Starvation

Don’t pick the same victim every time — add its rollback count to the cost.

APPLIED TO FIGURE 7(A)

Choose **T2** as the victim (e.g. youngest, little work done) and roll it back:

T2 releases S(Y) ▷ T3 gets X(Y), finishes, frees Z ▷ T1 gets X(Z), finishes ▷ T2 restarts

Wait-Die vs. Wound-Wait

Wait-Die

Non-preemptive

Older asks, younger holds

Older WAITS

Younger asks, older holds

Younger DIES — rolled back

“Old waits, young dies.”

Wound-Wait

Preemptive

Older asks, younger holds

Older WOUNDS younger — younger rolled back

Younger asks, older holds

Younger WAITS

“Old wounds, young waits.”

Both: the older transaction always wins, and a rolled-back transaction restarts with its **original timestamp** — so no starvation.

LOG-BASED RECOVERY

Why Do We Need a Log?

A crash removes main memory. The **log** — kept on stable storage — records every change, so we can repair the database afterwards.

Transaction Failure

Logic error or deadlock victim. Undo just that transaction.

System Crash

Power or OS failure. Memory is lost, disk survives. **Use the log.**

Disk Failure

The disk itself is damaged. Restore from a backup copy.

UNDO keeps **atomicity**: all or nothing

REDO keeps **durability**: commits survive

LOG-BASED RECOVERY

Reading a Log Record

T1	X	500	450
transaction	data item	old value → **UNDO**	new value → **REDO**

`<T1, X, 500, 450>` means “T1 changed X from 500 to 450”

Record	Meaning
<Ti start>	Ti has begun
<Ti commit> / <Ti abort>	Ti finished: committed, or its rollback is complete
<Ti, X, V>	Redo-only: written while undoing — “X was set back to V”
<checkpoint L>	L = transactions still active at the checkpoint

LOG-BASED RECOVERY

Deferred vs. Immediate Modification

Deferred

Changes reach the database **only after commit**.

- Crash before commit: nothing to fix
- Needs **REDO only**, never undo
- Log keeps only the new value

Immediate

Changes may reach the database **before commit**.

- Crash may leave half-done work
- Needs **UNDO and REDO**
- Log keeps old and new value

WRITE-AHEAD LOGGING (WAL) RULE

The **log record goes to stable storage first**, then the data change may go to disk.

LOG-BASED RECOVERY

The Golden Rule: Redo or Undo?

Log has **<Ti start>** AND **<Ti commit>** or **<Ti abort>**

→ REDO Ti

Replay its updates using the **new** values.

Log has **<Ti start>** but **no commit or abort**

→ UNDO Ti

Restore **old** values, then log <Ti abort>.

QUICK CHECK

`<T0 start> <T0,A,100,50> <T0 commit> <T1 start> <T1,B,20,30> CRASH`

T0 committed → **REDO** (A = 50). T1 never finished → **UNDO** (B = 20).

Why redo an aborted one? Its rollback was logged as redo-only records — redoing them simply repeats the rollback.

LOG-BASED RECOVERY

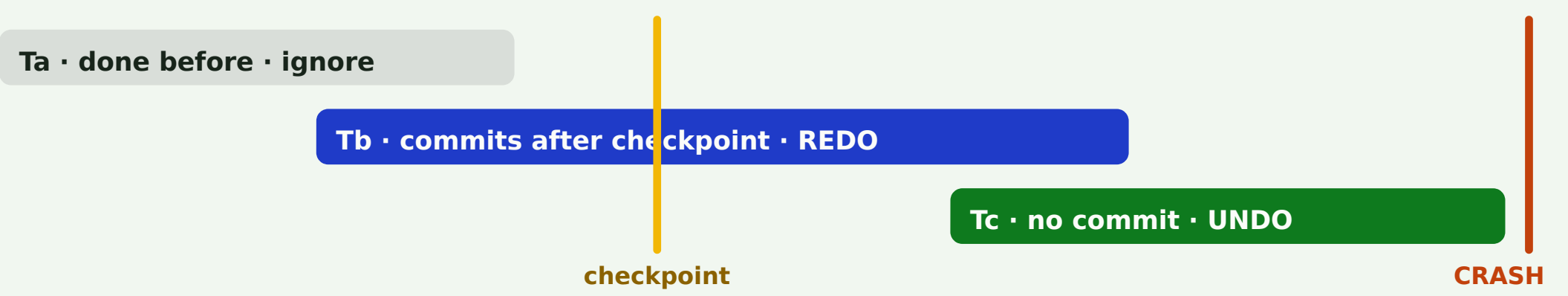
Checkpoints: A Safe Starting Point

THE PROBLEM

Without checkpoints, recovery scans the **whole log** and redoes work already safe on disk. Slow.

TAKING A CHECKPOINT

1. Write all log records in memory to stable storage
2. Write all modified buffer blocks to disk
3. Write **<checkpoint L>**, L = active transactions



After a crash, start from the **last checkpoint**. Only transactions in L, or started later, need any work.

LOG-BASED RECOVERY

The Recovery Algorithm in Two Passes

1 · Redo Phase

Scan FORWARD from the last checkpoint

1. Start: undo-list = L from <checkpoint L>
2. <Ti, X, V1, V2> → set X = V2
3. <Ti, X, V> → set X = V
4. <Ti start> → add Ti to undo-list
5. <Ti commit> or <Ti abort> → remove Ti

2 · Undo Phase

Scan BACKWARD from the end of the log

1. Update by Ti in undo-list → set X = V1, write <Ti, X, V1>
2. <Ti start> of Ti in undo-list → write <Ti abort>, remove Ti
3. Stop when undo-list is empty

What Does This Log Tell Us?

```
<T0 start>
<T0, Y, 1000, 1500>
<T0 commit>
<T1 start>
<T1, Y, 1500, 1200>
<T2 start>
<T4 start>
<T2, Z, 400, 300>
<checkpoint {T1, T4, T2}>
<T2, B, 600, 650>
<T3 start>
<T3, X, 600, 700>
<T1, Y, 1500>
<T4, A, 300, 200>
<T1 abort>
— SYSTEM CRASH —
```

T0 committed before the checkpoint → already safe on disk. **Ignore.**

Checkpoint: T1, T4, T2 active → start with **undo-list = {T1, T2, T4}**.

T3 starts after the checkpoint — the redo pass will add it.

<T1, Y, 1500> is redo-only: T1 was rolling back, and **<T1 abort>** closes it.

T2, T3, T4 have no commit or abort → they will be **undone**.

Redo Phase — Scan Forward

Log record	Action	Undo-list after
<checkpoint {T1,T4,T2}>	Start here: list = L	{T1, T2, T4}
<T2, B, 600, 650>	B = 650	{T1, T2, T4}
<T3 start>	add T3	{T1, T2, T3, T4}
<T3, X, 600, 700>	X = 700	{T1, T2, T3, T4}
<T1, Y, 1500>	Y = 1500 (redo-only)	{T1, T2, T3, T4}
<T4, A, 300, 200>	A = 200	{T1, T2, T3, T4}
<T1 abort>	remove T1	{T2, T3, T4}

End of log reached. Transactions to undo: **T2, T3, T4**. No new log records are written in this pass.

Undo Phase — Scan Backward

Log record (newest first)	Action	New log record
<T1 abort>	T1 not in list — skip	—
<T4, A, 300, 200>	A = 300	<T4, A, 300>
<T1, Y, 1500>	T1 not in list — skip	—
<T3, X, 600, 700>	X = 600	<T3, X, 600>
<T3 start>	remove T3 → {T2, T4}	<T3 abort>
<T2, B, 600, 650>	B = 600	<T2, B, 600>
<checkpoint {T1,T4,T2}>	list not empty — keep going	—
<T2, Z, 400, 300>	Z = 400	<T2, Z, 400>
<T4 start>	remove T4 → {T2}	<T4 abort>
<T2 start>	remove T2 → { } STOP	<T2 abort>

Recovery Log Records & Final Values

WRITTEN DURING RECOVERY

<T4, A, 300>

<T3, X, 600>

<T3 abort>

<T2, B, 600>

<T2, Z, 400>

<T4 abort>

<T2 abort>

DATABASE AFTER RECOVERY

A	B	X
300	600	600

Y	Z
1500	400

Every unfinished change is gone. **Y = 1500** is T0’s committed value — T1’s change was already rolled back.

ARIES: The Industry-Standard Recovery

Algorithms for **R**ecovery and **I**solation **E**xploiting **S**emantics — used by real systems such as IBM DB2 and SQL Server.

PHASE 1

Analysis

What was going on at the crash?

PHASE 2

Redo

Repeat history up to the crash

PHASE 3

Undo

Roll back the losers

`LSN`

A unique number on every log record

`PageLSN`

Last update already applied to a page

`Dirty Page Table`

Pages changed in memory, not yet on disk

`CLR`

Compensation record written for each undo

ARIES Step by Step

1 · Analysis

- Start at the last checkpoint, scan forward
- Rebuild the transaction table and dirty page table
- RedoLSN = smallest RecLSN in the dirty page table
- No commit/end → a **loser** (undo-list)

2 · Redo

- Scan forward from RedoLSN
- Reapply an update only if the page is dirty and PageLSN < its LSN
- **Repeats history** for all transactions, losers too

3 · Undo

- Scan backward, undo every loser’s updates
- Write a **CLR** for each undo; its UndoNextLSN skips finished work
- Write <Ti abort> / end when a loser is done

Who Needs REDO, Who Needs UNDO?

TABLE 8(a)

```
<T1 start>
<T1, X, 500, 450>
<T2 start>
<T2, Y, 300, 250>
<T1, Z, 800, 900>
<T1 commit>
<T2, W, 1000, 950>
– CRASH –
```

T1 → REDO

Has <T1 start> **and** <T1 commit>. Its work must survive (durability).

T2 → UNDO

Has <T2 start> but **no commit**. Its work must vanish (atomicity).

Recovery Actions, Step by Step

1 · Analysis

T1 has a commit → **winner**

T2 has no commit → **loser**

```
undo-list = {T2}
```

2 · Redo (forward)

Repeat history:

```
X = 450 (T1)
Y = 250 (T2)
Z = 900 (T1)
W = 950 (T2)
```

3 · Undo (backward)

<T2, W, 1000, 950> → **W = 1000**, log <T2, W, 1000>

<T2, Y, 300, 250> → **Y = 300**, log <T2, Y, 300>

<T2 start> → log **<T2 abort>**, done

Short version: redo T1 (X = 450, Z = 900), undo T2 (W = 1000, Y = 300). Same final result.

Final Values & New Log Entries

(III) DATABASE AFTER RECOVERY

X

450

T1’s value — kept

Y

300

T2 undone — restored

Z

900

T1’s value — kept

W

1000

T2 undone — restored

(IV) NEW LOG ENTRIES, IN ORDER

```
<T2, W, 1000>
<T2, Y, 300>
<T2 abort>
```

The first two are redo-only records. In ARIES they are called **CLRs** (compensation log records) and are never undone.

Ensuring Secure User Authentication

Store Passwords Safely

Never plain text. Save a salted hash (bcrypt, Argon2).

Multi-Factor Login

Password plus a second proof: OTP, app code or key.

Protect the Session

Encrypt traffic (TLS/HTTPS); session tokens that expire.

Limit and Watch

Lock out after failed tries, least-privilege roles, audit logs.